

DAY – 01: 90 Days of Devops

Linux Architecture, Processes, and systemd

Task

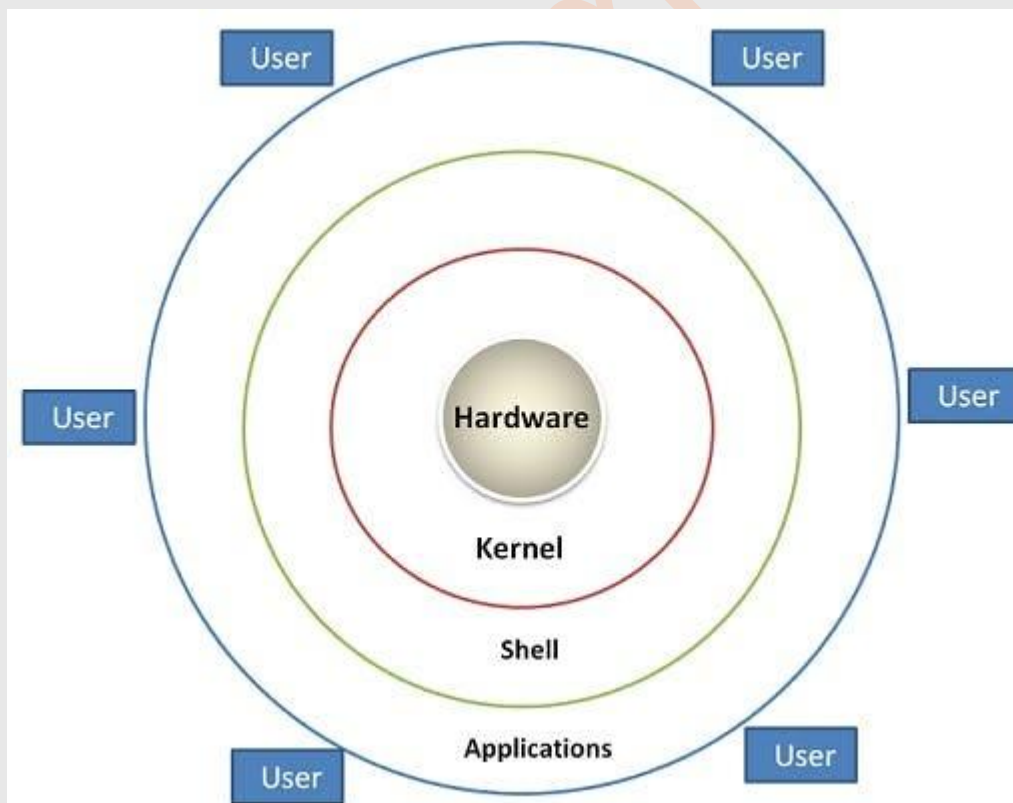
Today's goal is to **understand how Linux works under the hood.**

will create a short note that explains:

- The core components of Linux (kernel, user space, init/systemd)
- How processes are created and managed
- What systemd does and why it matters

This is the foundation for all troubleshooting you will do as a DevOps engineer.

Linux Architecture



Linux is built in layers, each with a specific role — from hardware control to user interaction.

1. Hardware Layer

- The physical components: CPU, memory, disks, and devices.
- Linux interacts with these through device drivers.

2. Kernel Layer

- The **core** of Linux — it manages communication between hardware and software.
- Key functions:
 - **Process Management** – handles creation, scheduling, and termination.
 - **Memory Management** – allocates and frees memory.
 - **Device Management** – controls hardware via drivers.
 - **File System Management** – organizes data into files and directories.

3. System Library Layer

- Provides standard functions (like `printf`, `open`, `read`) that applications use to interact with the kernel.
- Example: **glibc** (GNU C Library).

4. System Utility Layer

- Contains essential tools for system maintenance and configuration.
- Examples: `ls`, `cp`, `ps`, `systemctl`.

5. User Layer

- Where users interact with the system through:
 - **Shells** (e.g., `Bash`, `Zsh`)
 - **Graphical Interfaces** (e.g., `GNOME`, `KDE`)
 - **Applications** (e.g., browsers, editors)

Explanation of the Layers:

1. Hardware – The physical components (CPU, memory, storage, devices).
2. Kernel – The core of Linux that manages processes, memory, devices, and file systems.
3. System Libraries – Provide APIs that let programs talk to the kernel (e.g., `glibc`).
4. System Utilities – Essential tools for system configuration and management.
5. User Applications – Shells, GUIs, and programs that users interact with directly.

Core Components of Linux

- **Kernel:** The heart of the operating system. It manages hardware resources (CPU, memory, devices) and provides system calls for user applications.
- **User Space:** Where applications and utilities run. This includes shells, graphical environments, and user programs.
- **Init/Systemd:** The first process started by the kernel after boot. Traditionally `init`, but now most modern distributions use **systemd** to initialize and manage services.

How Processes Work

- A process is just a running program.

- Linux creates new processes by **copying an existing one** (fork) and then loading a new program into it (exec).
- Each process has a unique ID (PID) and is scheduled by the kernel.
- You can manage them with commands like ps, top, kill, or systemctl.

Why Systemd Matters

- It **starts services in parallel** → faster boot.
- It **manages dependencies** → ensures things start in the right order.
- It gives you **one tool** (systemctl) to control services, timers, sockets, and more.
- It integrates logging with **journald**.

In short: **Systemd makes Linux startup and service management smoother, faster, and more consistent.**

Simple Analogy

Think of Linux like a car:

- **Hardware** → Engine, wheels, body
- **Kernel** → The driver controlling everything
- **System Libraries** → The dashboard and controls
- **Utilities** → Tools for maintenance
- **User Layer** → You, giving commands to drive